

PipelineQ

Mini Hiring Pipeline Manager — Project Submission

GITHUB REPO

github.com/rnk3120/Mini_Hiring_Pipeline

STACK

HTML · CSS · Vanilla JS · localStorage

DEPENDENCIES

Zero — open index.html directly

DATE

27 September 2026

01 — OVERVIEW

What this is

PipelineQ is a single-page hiring pipeline manager built for a single recruiter managing candidates for one job opening. Candidates move through five ordered stages with full immutable history and a smart search bar that understands plain English.



↳ Reject is available at any stage before Hired. **Rejected** is also a terminal state.

02 — ARCHITECTURE

How it's built

The application is three files with no build step, no bundler, and no external dependencies. It works by opening `index.html` directly in a browser.

STATE LAYER

localStorage stores all data as a JSON snapshot. Reads reconstruct the full state on load. Perfectly sufficient for a single-recruiter, single-device use case.

AUDIT TRAIL

History entries are **append-only**. Every mutation creates a new array: `[...old, newEntry]`. No edit function exists. Timestamps are set once at write time and never touched again.

SEARCH ENGINE — LAYER 1

NLP intent parser (regex-based). Extracts structured intents: stage filter, stuck-for-duration, moved-since-date, offer-not-hired, stage exclusion. Returns descriptive errors on bad input.

SEARCH ENGINE — LAYER 2

Levenshtein fuzzy match (custom 20-line implementation). Handles typos in candidate names ("sharam" → "Sharma"). Substring match scores 1.0; word-level edit distance scales below that.

STAGE RULES

UI

Stages are an **ordered array**. `nextStage()` returns the next index — skipping is structurally impossible. `Hired` and `Rejected` are terminal; no advancement function exists for them.

Vanilla JS DOM. Kanban board with 6 columns, search-results view, three modals (add, detail, reject), toast notifications. All rendering is functional: state → DOM.

File structure

```
pipelineiq/
├─ index.html      App shell, semantic HTML, all modals
├─ styles.css      Design system – tokens, layout, components
├─ app.js          All logic: state, search, actions, rendering
├─ README.md       How to run, decisions, what's next
└─ ai_chat_logs.md AI conversation log with the Fuse.js disagreement
```

03 — SMART SEARCH

What the search can do

The search bar accepts plain English. A two-layer engine handles both structured queries and fuzzy name lookups. Multiple intents can be combined in one query.

QUERY	WHAT IT DOES
Priya	Fuzzy name search — finds "Priya Sharma" exactly
sharam	Levenshtein match — finds "Priya Sharma" despite the typo
in Interview	All candidates currently in the Interview stage
who's in Screening	Same — natural language alias handled
stuck in Screening for more than a week	Candidates in Screening ≥ 7 days
moved to Interview since Monday	Candidates who entered Interview in the last N days
offer not hired	Candidates who reached Offer but aren't Hired
everyone except rejected	All candidates excluding the Rejected stage
nonsense query	Descriptive error message — never silent empty result

04 — KEY DECISIONS

Why these choices

- 1 **Vanilla JS over React/Vue**
The problem is explicitly one recruiter, one job, one pipeline — a small, bounded CRUD problem. A framework would add ~100KB of runtime, a build pipeline, and module complexity for zero functional benefit. The app loads instantly and works offline.
- 2 **localStorage over a backend**
A single-recruiter tool needs no server. localStorage gives instant reads/writes, survives restarts, and has zero infrastructure cost. The tradeoff is accepted: for a team product, I'd use Supabase + Postgres with optimistic UI.
- 3 **Append-only history as the immutability primitive**

Rather than a database constraint, immutability is enforced structurally in code: the only mutation API is `addHistoryEntry()`. The pattern `c.history = [...c.history, newEntry]` makes it impossible to alter existing entries even accidentally.

4 Regex intent parser over a real NLP library

For 6–7 distinct query patterns, a lightweight regex parser is the right tool. A full NLP library (compromise.js, natural) would be overkill, add a dependency, and make the error messages harder to control. The parser is ~80 lines and covers all required query types.

⚠ WHERE I DISAGREED WITH THE AI

AI recommended: Use Fuse.js via CDN for fuzzy search (`<script src="https://cdn.jsdelivr.net/.../fuse.min.js">`).

My decision: I wrote a custom 20-line Levenshtein implementation instead. Two reasons: (1) A CDN dependency makes the app fail offline — this is a static file that should work anywhere. (2) Fuse.js's scoring is opaque; a custom implementation gives complete control over ranking logic and makes it easy to debug why a result ranks where it does. The cost was 20 lines of well-understood code. The benefit was zero dependencies and offline safety.

05 — DESIGN

Interface approach

The design takes deliberate cues from Linear and Vercel's design language — dark, restrained, typographically precise. Every color choice serves a functional purpose: stage accent colors communicate meaning, not decoration. Spacing follows an 8px baseline grid.

COLOR PALETTE

Dark surfaces — `#0e0e10` base, three elevation levels. Stage colors only appear on meaningful elements (dots, chips, timeline). No gradients, no glow.

TYPOGRAPHY

Inter Variable at 13px base. Three weights (400, 500, 600). Labels at 11px / 0.08em tracking. Names at −0.01em for tight, readable density.

KANBAN COLUMNS

Each column has a **2px accent line** at the top (stage colour) and a neutral header. Cards carry a top-border divider — no heavy card shadows or glassmorphism.

INTERACTION

Cards lift **1px on hover** with a 120ms ease. Modals scale from 0.98 → 1 in 220ms. All animations use `cubic-bezier(0.16, 1, 0.3, 1)` — the "spring" feel without a library.

06 — WITH MORE TIME

What I'd add next

📄 Backend + Database

Supabase (Postgres) to replace localStorage. Enables multi-user, cross-device access.

⚡ Real-time updates

WebSocket / SSE so two recruiters see live pipeline changes without refresh.

🧪 Unit tests

Vitest coverage for `parseQuery()`, `advanceCandidate()`, and audit trail immutability.

🔍 Smarter search

Phonetic matching, date range queries ("applied in September"), role-based filters.



Analytics

Time-to-hire, stage conversion rates, avg days per stage — recruit smarter over time.



Accessibility audit

Full keyboard navigation, ARIA live regions for search, high-contrast mode.

PipelineIQ

GitHub: github.com/YOUR_USERNAME/pipelineiq